

Terraform / IaC Security Scanner Benchmark on Labelled AWS Terraform Modules

Vishvaksen Machana | Student ID: 25173421

MSc Cloud Computing — National College of Ireland — Short Explanation Guide

Baseline: Verdet et al. (2025) Empir. Softw. Eng. — DOI 10.1007/s10664-024-10610-0

Read aloud in simple English. Numbers come from STATUS.md / results in the repo — do not invent live AWS metrics in class.

1. What is the project?

This project benchmarks how well popular Infrastructure-as-Code (IaC) security tools find known misconfigurations in AWS Terraform modules. Cloud teams write Terraform to create buckets, roles, networks, and databases. Before that code reaches production, scanners and policy gates are supposed to catch dangerous settings. The open question is not whether the tools raise alerts — it is what fraction of known labelled defects they actually identify, with precision and recall measured the same way for every stage.

The formal artefact is terraform-scanner-benchmark/ only. Do not present the quarantined War hybrid under _superseded_proxy/iac-security/ as evidence. I built a labelled corpus of N=240 AWS Terraform modules (60 per category × 4 categories; about 60% defective). Each module carries a published secure/insecure ground-truth label in corpus/labels.csv. Three detection stages are scored against those labels: (1) a deterministic label-oracle checklist aligned to the published labels, (2) static scanners Checkov 3.3.19 and tfsec v1.28.14 at shipped defaults, plus their static union, and (3) an OPA/Rego 1.4.2 policy-as-code CI gate with category-scoped packages under policies/regol/.

Ethics are binding for the whole thesis: evaluation-only corpus — no terraform apply and no live AWS infrastructure is created from these modules. Tool versions are pinned in results/tool_versions.json so a reader can reproduce the same binaries. CA2 floor status is COMPLETE at 100% research alignment. One-sentence pitch: “I measure precision, recall, and F1 of Checkov, tfsec, and an OPA gate against labelled AWS Terraform defects — without ever applying the modules to a real cloud account.”

2. Problem statement + real-life example

Misconfigured cloud infrastructure is a common breach path: public storage, world-open security groups, missing encryption at rest, and disabled logging. Terraform makes those settings code, so defects can spread through pull requests if reviewers rely only on gut feel. Checkov and tfsec encode hundreds of checks; OPA lets a team write custom Rego policies as a merge gate. But scanners disagree on the same file, and a tool with excellent precision can still miss a large share of known defects (low recall). Without a labelled oracle, “we ran Checkov in CI” is not a measured security claim — it is only a process statement.

Real-life example — building inspection before opening a street of shops. Terraform files are the blueprints. Checkov and tfsec are two different city inspectors, each with a printed checklist of fire, wiring, and access rules. OPA is the permit desk that can block the opening if a custom city policy fails. My labelled corpus is a street of 240 shops where I already know which ones have faulty wiring, unlocked back doors, missing fire alarms, or no CCTV logs. I ask each inspector, and the permit desk, what fraction of those known faults they catch on the exact same street. Verdet et al. studied how often developers adopt security policies across cloud providers; I measure scanner and OPA recall on known AWS Terraform defects.

Why it is hard in practice: Checkov and tfsec map different check IDs to similar ideas; alert conflicts need paired statistical tests rather than anecdotal screenshots; high OPA precision can hide incomplete category coverage; and applying insecure modules to live AWS would be unethical for a student thesis — so scoring must stay local against labels. Transfer of these results to much larger public Terraform repos is soft optional work, not a CA2 blocker.

3. Baseline paper

Verdet, Hamdaqa, Da Silva and Khomh (2025), “Assessing the adoption of security policies by developers in Terraform across different cloud providers”, *Empirical Software Engineering* 30(3), 74 (DOI 10.1007/s10664-024-10610-0). They examine security-policy adoption in real Terraform projects across cloud providers and discuss Checkov/tfsec behaviour, including accuracy analyses that focus on disputed or conflicting cases rather than a full labelled recall study on a purpose-built AWS module corpus.

Their gap for this thesis: no published Terraform study reports recall, precision, and F1 against labelled ground truth across an oracle/manual checklist, Checkov, tfsec, and an OPA gate on the identical module set, while also recording scan time and remediation effort. Rahman-style and GLITCH-style labelled IaC smell benchmarks are cited as oracle-methodology anchors — they are not a substitute Terraform scanner baseline. The older War et al. comment-ablation proxy is superseded for CA2 and must not be named as the binding gap paper in a viva.

How we connect to Verdet honestly: we keep their interest in Checkov/tfsec disagreement and add McNemar tests with Holm–Bonferroni correction on pre-registered pairs (docs/VERDET_COMPARISON.md), while the primary contribution is labelled per-category P/R/F1 on the N=240 corpus. We do not claim to re-run Verdet's multi-cloud adoption crawl.

4. How we solve it / research question

Research question: What percentage of the labelled AWS misconfigurations will be identified by Terraform security scanners and an enforced policy-as-code gate?

Solution approach: freeze pinned scanner and OPA versions; score every stage against corpus/labels.csv; report precision, recall, F1, and false-negative rate per category and overall; record scan time per stratified module sample and remediation LOC (secure↔insecure diff). Categories follow prevalence themes used in the Verdet/Buhler literature: public_storage, overpermissive_access, encryption_at_rest, and weak_logging. A-priori check-id maps under mappings/ stop post-hoc cherry-picking of which Checkov or tfsec rule “counts”.

Honest STATUS / metrics_per_category.csv numbers (do not invent others): label-oracle checklist F1 = 0.876 (precision 0.951, recall 0.812, identified 81.2% of labelled defects); Checkov F1 = 0.651 (recall 0.562); tfsec F1 = 0.693 (recall 0.618); static union F1 = 0.718 (recall 0.708); OPA gate F1 = 0.753 (precision 1.000, recall 0.604). Negatives retained on purpose for rubric honesty: Checkov/tfsec alone miss many labelled defects; OPA's perfect precision does not mean complete coverage of the labelled set.

5. Dataset / data (professor cares about this)

Purpose-built labelled Terraform modules. No live cloud telemetry. No customer PII. No production accounts. Synthetic labelled modules are explicitly allowed by the formal CA2 contract, because the research object is detection accuracy against known labels, not scraping someone else's private infrastructure.

- N=240 AWS Terraform modules: 60 per category × 4 categories; ≈60% labelled defective.
- Categories: public_storage (public ACL/policy, publicly_accessible, public AMI/IP); overpermissive_access (world CIDRs, IAM *, AdministratorAccess); encryption_at_rest (missing/disabled SSE); weak_logging (missing CloudTrail / VPC flow / access logs).
- Ground truth: corpus/labels.csv written with the modules (Rahman/GLITCH-style labelled oracle).
- Identical inputs for every stage: checklist, Checkov, tfsec, static union, OPA gate.
- Primary tables: results/metrics_per_category.csv, scan_times.csv, remediation_loc.csv, tool_versions.json, mcnemar_pairs.csv, holm_bonferroni.csv.
- Ethics: evaluation-only; AWS apply forbidden; evidence lives only under terraform-scanner-benchmark/results/.

6. Implementation + technologies / AWS services

- Checkov 3.3.19 — static IaC security scanner at shipped defaults.
- tfsec v1.28.14 — second static scanner; paired McNemar comparison versus Checkov.
- OPA 1.4.2 + Rego — category-scoped policy-as-code CI gate (policies/regos).
- Python 3 + python-hcl2 — generate corpus, run stages, evaluate, pytest harness.

- A-priori Checkov/tfsec check-id maps under mappings/ (not post-hoc cherry-picking).
- Terraform CLI may be present for syntax context only — never applied to AWS.
- Four AWS misconfiguration themes expressed as HCL modules (S3 / IAM / encryption / logging patterns).
- No live AWS account is required for the formal CA2 research floor.

7. End-to-end flow

- Generate or load the labelled corpus (scripts/generate_corpus.py → corpus/).
- Run label-oracle checklist rules documented in docs/MANUAL_CHECKLIST.md versus labels.csv.
- Batch-scan corpus/ with pinned Checkov and tfsec; compute static union of findings.
- Evaluate OPA/Rego category packages as a CI gate on the same modules.
- Compute per-category and overall P/R/F1/FN → metrics_per_category.csv.
- Record stratified scan-time samples and remediation LOC diffs (secure↔insecure).
- Run McNemar + Holm–Bonferroni pairs for the Verdet-aligned comparison write-up.
- Archive results under terraform-scanner-benchmark/results/ — never terraform apply.

8. Exact speaking script (dataset + implementation + technologies)

“My dataset is two hundred and forty labelled AWS Terraform modules that I built for this thesis — sixty modules in each of four misconfiguration categories, with known secure or insecure labels stored in labels.csv. About sixty percent are defective on purpose so recall is measurable. I never applied any module to a real AWS account, because the ethics rule is evaluation-only. I implemented three detection stages on the exact same inputs: a deterministic label-oracle checklist, the static scanners Checkov 3.3.19 and tfsec v1.28.14 plus their union, and an OPA 1.4.2 Rego policy gate. I report precision, recall, and F1 per category, plus scan time and remediation lines of code. Technologies are Python, HCL parsing, Checkov, tfsec, and OPA. My baseline is Verdet et al. 2025 on Terraform security-policy adoption; the gap I fill is labelled recall and F1 across checklist, scanners, and OPA — not only alert conflicts on disputed cases. The old War hybrid proxy is not my evidence.”

9. STATUS snapshot + five likely Q&A

STATUS snapshot (say this if asked “are you finished?”): INITIAL_EVAL_PASS=yes; final_1|2|3 DONE; CA2 floor COMPLETE; ALIGNMENT=100; AWS_APPLY=forbidden; ORACLE=labelled_corpus; War hybrid = PROXY only.

Q: Did you terraform apply these modules on AWS?

A: No — ethics forbid apply; all scoring is local against labelled ground truth.

Q: Is iac-security/ or the War hybrid your artefact?

A: No — that tree is the superseded proxy. Formal evidence is only terraform-scanner-benchmark/.

Q: Which scanners and versions did you pin?

A: Checkov 3.3.19, tfsec v1.28.14, OPA 1.4.2 — recorded in tool_versions.json.

Q: Did Checkov catch all labelled defects?

A: No. Checkov recall is 0.562; static union recall 0.708; OPA has precision 1.0 but recall 0.604.

Q: What exactly is the Verdet gap you claim?

A: Verdet does not report labelled recall/F1 across checklist, Checkov/tfsec, and OPA on the same Terraform modules.